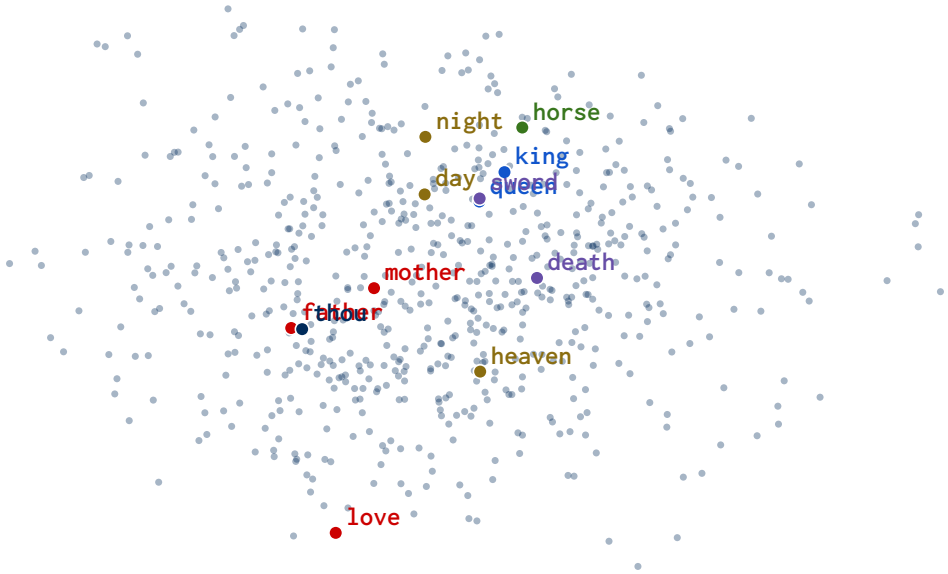


UNIT 01 · CHAPTER 6

Text Embeddings

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Token IDs

In the last chapter irishQwen's tokenizer learned **253** merges from Shakespeare and turned Hamlet's "To be, or not to be" into token ids, `446` for "To", `309` for " be" and so on. As we saw there, those ids are labels and nothing more. The tokenizer hands the model a list of jersey numbers and **the model has to work out what each one means**.

A network can't do much with a jersey number. Everything a network does is multiply and add, and multiplying "the token with id **446**" by a weight treats it as if it were **446** of something. What we need is a way to turn each id into numbers that a network can multiply and add, and that say something about the token. Pictures had the same problem, and we already know how networks deal with it there.

Word Tokens

For this chapter we'll split Shakespeare into whole words rather than BPE tokens. It's much easier to talk about what "king" means than what "qu" means (irishQwen cuts "queen" into "qu", "e" and "en"), and everything we do works the same way on any kind of token. Here is a word vocabulary built from the first million characters of the corpus, commonest word first:

```
import re
from collections import Counter
text = open("shakespeare.txt").read()[:1_000_000]
words = re.findall(r"[a-z']+", text.lower())
vocab = [w for w, c in Counter(words).most_common()]
ids = {w: i for i, w in enumerate(vocab)}
prompt = "to be or not to be".split()
print(len(words), "words,", len(vocab), "different")
print([ids[w] for w in prompt])
print(ids["king"], ids["queen"], ids["day"], ids["night"])
```

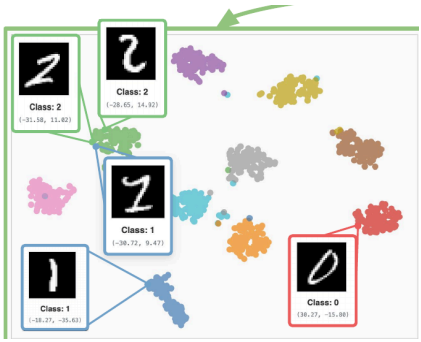
OUTPUT

```
166796 words, 11774 different
[3, 18, 39, 11, 3, 18]
139 211 156 215
```

"to" is word **3** and "be" word **18**, so the line is the list `[3, 18, 39, 11, 3, 18]`. Notice that the numbers say nothing useful about the words. "king" (**139**) is closer to "day" (**156**) than "day" is to "night" (**215**), simply because of how often each word happens to turn up. Any arithmetic a network did on these ids directly would be arithmetic on word counts.

Image Embeddings

We've seen this problem before, with pictures. A network that reads the MNIST digits from High Dimensional Data gets **784** pixel brightnesses, and a pixel on its own means very little. After the first layer, though, each image has become a list of **256** numbers, and those numbers are where the network keeps what it has noticed about the image (a loop at the top, a stroke down the middle).



Pictured above are hidden-layer vectors for thousands of digits, squashed down to two dimensions so we can plot them. **Digits of the same class land near each other**, even when they're written quite differently. The **256** numbers are a description of the image that the network learned for itself, and an **embedding** is exactly that, a list of numbers that stands in for a thing, where similar things get similar lists.

Our tokens need the same treatment. The question is what to start from, since a token id is even less informative than a pixel.

One-Hot Vectors

The simplest way to turn an id into a vector is a **one-hot vector**: a list as long as the vocabulary, all zeros except for a single 1 at the token's id. "to" becomes `11,774` numbers with a 1 in position `3`. PyTorch will do this for a whole prompt at once:

```
import torch
import torch.nn.functional as F
# "to be or not to be" as word ids
t = torch.tensor([3, 18, 39, 11, 3, 18])
T = F.one_hot(t, num_classes=11774)
print(T.shape)
print(T[:, :8])
print((T[0] * T[1]).sum(), (T[0] * T[4]).sum())
```

OUTPUT

```
torch.Size([6, 11774])
tensor([[0, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0]])
tensor(0) tensor(1)
```

Each row is one word of the prompt (only the first `8` columns are printed), and the two copies of "to" get identical rows. Unlike the ids, one-hot rows don't pretend some words are closer than others. Multiplying two rows together and adding up gives `0` for different words and `1` for the same word.

Embeddings

Unfortunately that is also the problem with one-hot vectors. **Every pair of different words is exactly as far apart as every other pair**, so "day" is no more like "night" than it is like "sword". The vectors are also absurdly long, `11,774` numbers for our small vocabulary and `151,936` for the real Qwen3 tokenizer, almost all of them zero.

A **text embedding** gives every token a short list of ordinary numbers instead, the same length for every token. The length is the **embedding dimension**, written d. The slides use d = `768` (the size GPT-2 used), Qwen3-0.6B uses `1,024` and irishQwen's Shakespeare model uses `256`.

All of the embeddings sit in one table, the **embedding matrix** E, with one row per token in the vocabulary and d columns. Row `3` is the embedding of "to", row `18` the embedding of "be". The numbers in the table are weights, learned like any other weights in a network, which is why the book colours them blue.

Embedding Dimensions

What do the d numbers stand for? One way to think about it is that **each column measures some aspect of language**, plurality, colour, gender, whether a word is a verb, and two tokens that are close in a column are similar in that one respect. This is extremely approximate (the "aspects" are usually spread across many columns at once, and none of them come with labels), but it's a good way to think about it. The point is that embeddings, unlike ids or one-hot vectors, can put "day" near "night".

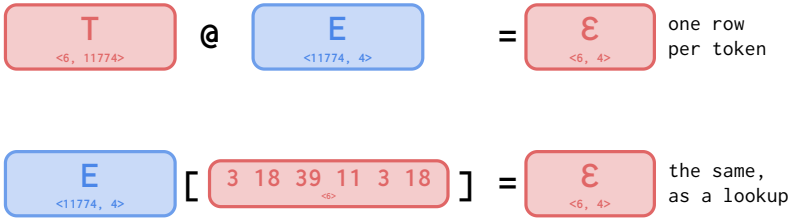
Notice that d is a choice, made when the model is built, and it doesn't depend on the vocabulary at all. A vocabulary of `512` tokens or `151,936`, every token still gets d numbers. Bigger models use bigger d because they have more to say about each token, and the cost is a bigger table.

The Embedding Layer

Getting a prompt's embeddings out of E is one matrix product. Stack the prompt's one-hot rows into a matrix T, with one row per token, and multiply by the table:

one-hot prompt times the embedding matrix

T@E = Ɛ



Each row of T has a single 1, so each row of the product is just one row of E, the row for that token. The matrix product is really **a lookup in a table**, and indexing E with the ids gives the same answer.

Lookups in Code

Here are both versions in PyTorch, with a random `E` of `4` numbers per word. `@` multiplies the one-hot rows by the table, and `E[t]` looks the rows up directly:

```
import torch
import torch.nn.functional as F
# "to be or not to be" as word ids
t = torch.tensor([3, 18, 39, 11, 3, 18])
T = F.one_hot(t, num_classes=11774).float()
torch.manual_seed(0)
E = torch.randn(11774, 4)   # 4 numbers per word
out = T @ E
print(out)
print(torch.equal(out, E[t]))
```

OUTPUT

```
tensor([[-2.3250, -0.2188, -1.2459, -0.7323],
        [ 1.0040, -0.6179,  1.8220, -1.3204],
        [ 0.3007, -0.1061, -1.1857, -2.3982],
        [ 0.2645, -0.3139,  1.4580,  1.9603],
        [-2.3250, -0.2188, -1.2459, -0.7323],
        [ 1.0040, -0.6179,  1.8220, -1.3204]])
tensor(True)
```

The two "to" rows and the two "be" rows match, as they should. A token's embedding doesn't depend on where it sits in the prompt, or on the tokens around it. "to" gets row `3` wherever it turns up.

Embedding Layers in irishQwen

Real models never build **T**. For Qwen3 each one-hot row would hold **151,936** numbers, and multiplying all those zeros by **E** would waste almost all of the work. So an **embedding layer** skips straight to the lookup. irishQwen's `EmbeddingLayer` keeps its weights as `W` with one column per token (the same layout as its `LinearLayer`), picks out the prompt's columns and moves the embedding to the last axis:

```
import torch
torch.manual_seed(0)
V, d = 512, 256           # irishQwen's Shakespeare model
W = torch.randn(d, V)     # one column per token
# "To be, or not to be" with irishQwen's tokenizer
tokens = torch.tensor([[446, 309, 44, 32,
                        275, 327, 294, 309]])
out = W[:, tokens]                  # (d, batch, seq)
out = out.permute(1, 2, 0)          # (batch, seq, d)
print(tokens.shape, "->", out.shape)
print(torch.equal(out[0, 1], out[0, 7]))   # " be" twice
```

OUTPUT

```
torch.Size([1, 8]) -> torch.Size([1, 8, 256])
tensor(True)
```

A batch of one prompt of **8** tokens becomes a tensor of shape `(1, 8, 256)`, and " be", which appears twice, gets the same **256** numbers both times. PyTorch's own version of this layer is `nn.Embedding`.

Random Embeddings

The table is big. Qwen3-0.6B's has `151,936` × `1,024` = `155,582,464` weights, **about a quarter of the whole model**, before it has done any thinking at all. irishQwen's Shakespeare model gets away with `512` × `256` = `131,072`.

Like every other layer, an embedding layer starts out random (irishQwen draws its weights from a normal distribution). A random table gives every token a different row, which is something, but the rows carry no meaning. In our `4`-number table from page 9, "to" and "be" are no more alike than "to" and "not", and "day" is as likely to land near "sword" as near "night".

So the numbers have to be learned. In irishQwen they are learned along with everything else, by the optimizer, while the model learns to predict the next token. That is a long way off (there's a whole transformer between the embeddings and the prediction), so for the rest of this chapter we'll train embeddings on their own, with a much smaller task. The question is what the task should be. Nobody hands us a list of which Shakespeare words mean similar things.

Co-occurrence

What we do have is a lot of text, which tells us which words appear near which. Here is how often six words we picked turn up within two places of "day", "night" and "sword" in our slice:

```python
import re
from collections import Counter
text = open("shakespeare.txt").read()[:1_000_000]
words = re.findall(r"[a-z']+", text.lower())
vocab = [w for w, c in Counter(words).most_common()]
ids = {w: i for i, w in enumerate(vocab)}
def neighbors(word):
    near = Counter()
    for i, w in enumerate(words):
        if w == word:   # the two words before and after
            near.update(words[max(0, i - 2):i])
            near.update(words[i + 1:i + 3])
    return near
cols = ["good", "this", "last", "every", "draw", "his"]
print("       ", cols)
for w in ["day", "night", "sword"]:
    near = neighbors(w)
    print(f"{w:>6}:", [near[c] for c in cols])
```

OUTPUT

```
        ['good', 'this', 'last', 'every', 'draw', 'his']
   day: [2, 18, 2, 8, 0, 0]
 night: [16, 8, 7, 1, 0, 0]
 sword: [1, 3, 0, 0, 8, 18]
```

"day" and "night" keep the same company ("good night", "this day", "last night") and neither is ever drawn, while a sword is drawn and is usually somebody's. These **co-occurrence** counts are already a crude embedding, with "day" near "night".

The Distributional Hypothesis

The idea that words used in similar places have similar meanings is called the **distributional hypothesis**. The linguist J. R. Firth put it as "**you shall know a word by the company it keeps**" (in 1957, so not a reference to Shakespeare, though it could be). Every method in this chapter, and the embeddings inside every large language model, rests on it.

Co-occurrence counts are a fine start, but they bring back the one-hot problem. A word's counts have one entry for every word in the vocabulary, `11,774` of them here, and most are zero. We want the same information squeezed into d numbers, and squeezing is something a network can learn.

Learning Embeddings

To get short vectors we can train an embedding layer on the company-keeping task directly. The best known way is **word2vec** (Mikolov and colleagues at Google, 2013): take a word and a word near it, and train the word's embedding so that its dot product with the neighbor's vector is high. Take the same word and a random word, and train the dot product to be low. A sigmoid turns each dot product into a probability, and the loss is cross-entropy, with two classes (real neighbor or random word).

Why would that give similar words similar embeddings? "day" and "night" both have to score high with "good", "this" and "last", and low with whatever random words come along. The easiest way for training to manage that is to **give "day" and "night" vectors that point the same way**. Nothing in the loss mentions meaning, similar embeddings fall out of similar company.

Word2Vec in PyTorch

Here is word2vec on the commonest **2,000** words of our slice. First the training data: every word is paired with each of the two words before and after it (shifting the whole text by one or two places does that in one go), and pairs with a rarer word are dropped:

```
import re
import torch
from collections import Counter
text = open("shakespeare.txt").read()[:1_000_000]
words = re.findall(r"[a-z']+", text.lower())
vocab = [w for w, c in Counter(words).most_common()]
ids = {w: i for i, w in enumerate(vocab)}
V = 2000                        # the 2,000 commonest words
t = torch.tensor([ids[w] for w in words])
word = torch.cat([t[2:-2]] * 4)                  # a word
near = torch.cat([t[:-4], t[1:-3], t[3:-1], t[4:]])
ok = (word < V) & (near < V)
word, near = word[ok], near[ok]
print(len(word), "pairs")
```

OUTPUT

```
524892 pairs
```

That gives over half a million (word, neighbor) pairs from a million characters of Shakespeare, all without anyone labelling anything.

Training Word2Vec

Training takes **512** real pairs and **512** random words at a time, and pushes the real dot products up and the random ones down:

```
import torch.nn as nn
import torch.nn.functional as F
torch.manual_seed(0)
vecs = nn.Embedding(V, 32)  # the embeddings we want
ctx = nn.Embedding(V, 32)   # helper: words as neighbors
params = [*vecs.parameters(), *ctx.parameters()]
opt = torch.optim.Adam(params, lr=0.01)
for step in range(301):
    i = torch.randint(0, len(word), (512,))
    fake = torch.randint(0, V, (512,))     # random words
    a = vecs(word[i])
    real = (a * ctx(near[i])).sum(1)   # push these up
    rand = (a * ctx(fake)).sum(1)      # push these down
    loss = -(F.logsigmoid(real)
             + F.logsigmoid(-rand)).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 100 == 0:
        print(step, round(loss.item(), 3))
```

OUTPUT

```
524892 pairs
0 4.43
100 3.469
200 2.514
300 1.944
```

`vecs` is the embedding layer we want, and `ctx` is a second table for words in their role as neighbors, which we throw away afterwards. The loss falls from **4.43** to **1.94** (the browser takes about half a minute).

Our Shakespeare Embedding

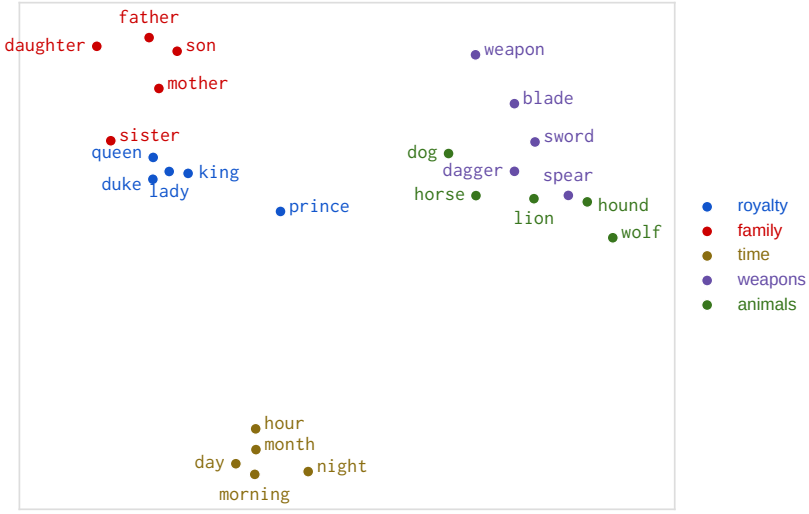
Three hundred steps on a slice isn't much. To get embeddings worth looking at we ran the same method on all of Shakespeare (**905,462** words) outside the browser, keeping the **9,022** words that appear at least **5** times and going over the text **15** times, with **64** numbers per word. That table is **our Shakespeare embedding**, and every similarity from here on comes from it. Here are the first **4** of the **64** numbers for a few words:

word	ε_1	ε_2	ε_3	ε_4	…
king	-0.56	0.27	-0.07	0.49	…
queen	-1.09	-0.10	0.38	0.36	…
day	0.09	-0.20	-0.12	-0.06	…
night	-0.21	0.06	0.01	-0.10	…
sword	-0.49	0.06	-0.42	-0.17	…

On their own the numbers mean nothing we can read off (no column turns out to be "royalty"), and it's hard to see from four columns that "king" and "queen" are alike. The similarity is in how all **64** line up, so to see it we have to look at whole vectors.

Embeddings as Points

An embedding of **64** numbers is a point in **64**-dimensional space, which is hard to draw. We can still get a feel for it by squashing a few words down to two dimensions with PCA (principal component analysis, which finds the two directions along which the points spread out the most):



The colors are groups we picked by hand, the embedding never saw them. Even so, **words of the same kind land together**. Time words sit at the bottom, family and royalty in the top left, weapons and animals on the right, where horses and hounds share the battlefield with swords.

Two dimensions throw away most of the **64**, so we can't measure anything off this picture. For that we need to compare the full vectors.

Euclidean Distance

In High Dimensional Data we compared football games two ways, and the first was the straight-line distance between them, the **Euclidean distance**: subtract the vectors, square each of the `64` differences, add them up and take the square root. In our Shakespeare embedding "king" is `2.38` from "queen" and `3.32` from "cat", which looks sensible.

Unfortunately it goes wrong quickly. "king" is `2.62` from "prince" but only `2.53` from "the", so by distance a king is more like "the" than like a prince. The reason is the lengths of the vectors. "the" is short (length `1.85`), "prince" is long (`2.84`), and a short vector is close to everything simply because it's close to the origin.

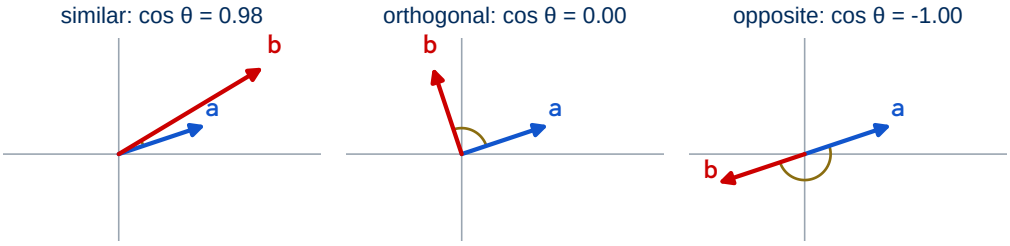
How long a word's vector ends up depends on things like how often the word appears and how its training went, which has little to do with meaning. **The direction a vector points is what carries the meaning**, so we want a measure that ignores length.

Cosine Similarity

The second way was **cosine similarity**, which measures the angle between two vectors and ignores their lengths. It is the dot product of the vectors divided by both of their lengths:

the angle between a and b, whatever their lengths

$$\cos\theta = \frac{a \cdot b}{\|a\| \, \|b\|}$$



Vectors pointing the same way score close to `1`, vectors at right angles score `0` and vectors pointing opposite ways score `-1`. Doubling a vector's length doesn't change its score at all.

Cosine Similarity in Code

Here are the three pairs from the picture, worked out from the formula and with PyTorch's built-in version:

```
import torch
import torch.nn.functional as F
a = torch.tensor([3., 1.])
for b in [[5., 3.], [-1., 3.], [-3., -1.]]:
    b = torch.tensor(b)
    cos = (a @ b) / (a.norm() * b.norm())
    same = F.cosine_similarity(a, b, dim=0)   # built in
    print(b.tolist(), round(cos.item(), 3),
          round(same.item(), 3))
```

OUTPUT

```
[5.0, 3.0] 0.976 0.976
[-1.0, 3.0] 0.0 0.0
[-3.0, -1.0] -1.0 -1.0
```

In our Shakespeare embedding "king" scores `0.638` with "queen", `0.524` with "prince", `0.349` with "the" and `0.12` with "cat", which is the order we'd hope for. Notice that unrelated words don't score `0` here. Two random words among the commonest `2,000` average about `0.27`, so **a score only means something compared with other scores**.

Nearest Neighbors

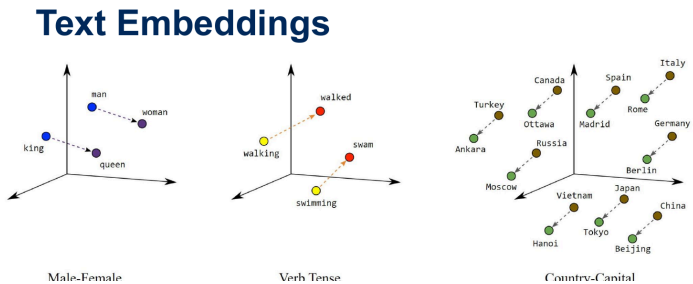
With a similarity measure we can ask the embedding which words it thinks are most like a given word: compare the word with all `9,022` others and keep the highest cosine similarities, its **nearest neighbors**.

word	its four nearest neighbors (cosine similarity)
queen	elizabeth 0.80, margaret 0.75, dowager 0.69, guildenstern 0.69
father	son 0.85, youngest 0.78, daughter 0.77, eldest 0.76
sword	sheathe 0.69, chop 0.68, he'd 0.64, blade 0.63
horse	hobby 0.63, barbary 0.62, hound 0.61, bestrid 0.58
thou	dost 0.77, wouldst 0.74, art 0.74, talk'st 0.73
day	wednesday 0.72, tuesday 0.71, wedding 0.71, twelvemonth 0.70

Most of these are pretty good. The queen's neighbors are actual queens (Elizabeth and Margaret in the history plays), "thou" comes with the verb forms that only go with "thou", and a horse keeps company with hounds, hobby-horses and Barbary (roan Barbary, the horse Bolingbroke rides in *Richard II*). A few are odd. Guildenstern is there because the stage directions keep bringing him on with the King and Queen, and every one of the Queen's speeches in Hamlet starts with the label "Queen." (the corpus is the plays as printed, labels and all). **Everything the embedding knows came from which words appeared near which**.

Directions in Embedding Space

Big embeddings trained on billions of words go further than neighbors. The difference between two related words points in a consistent direction, so "man" to "woman" is roughly the same step as "king" to "queen", and "walking" to "walked" the same step as "swimming" to "swam":



That means we can do arithmetic on meanings. Take "king", subtract "man", add "woman", and look for the word nearest the result. In the large word2vec embeddings the answer is famously "queen". Ours, trained on one (long) dead author, gets close. Its top answer is "bona" (`0.675`), Lady Bona, the French queen's sister whom Warwick tries to marry to Edward IV in *Henry VI, Part 3*, with "queen" second (`0.652`). "his" − "he" + "she" gives "her" (`0.777`), as it should.

Unfortunately most other analogies we tried in our small embedding come out as noise. **Clean directions need far more text than one author wrote**, which is one reason modern models train on so much of it.

Related Means Co-occurring

It's worth being precise about what similar embeddings mean. Two words get similar embeddings when they appear in similar company, and nothing else. Usually that lines up with similar meaning. Sometimes it doesn't:

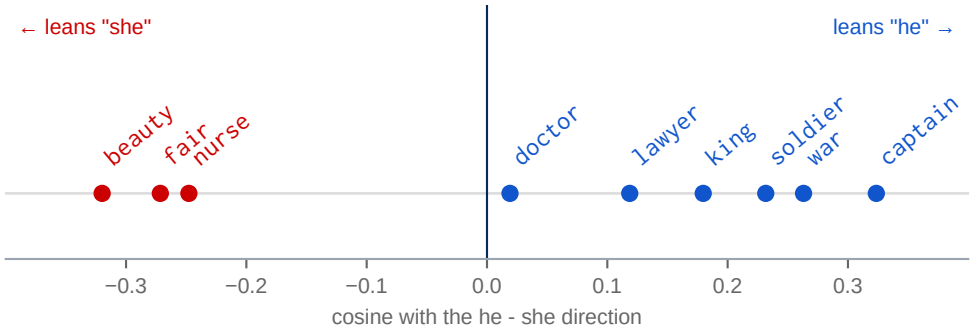
pair	cosine similarity
day, night	0.696
man, woman	0.669
clean, dirty	0.643
love, hate	0.637
good, bad	0.515
king, cat	0.120

Opposites score high, most of them higher than "king" and "prince" (`0.524`). "I love thee" and "I hate thee" are the same sentence with one word changed, so as far as company goes, **love and hate are nearly the same word**. Notice that this is also why ChatGPT can answer that the opposite of "dirty" is "clean": the two are close neighbors, and the model has learned from other text which neighbor counts as the opposite.

Remember that related only ever means co-occurring. An embedding is a summary of its training text, nothing more and nothing less.

Bias in Embeddings

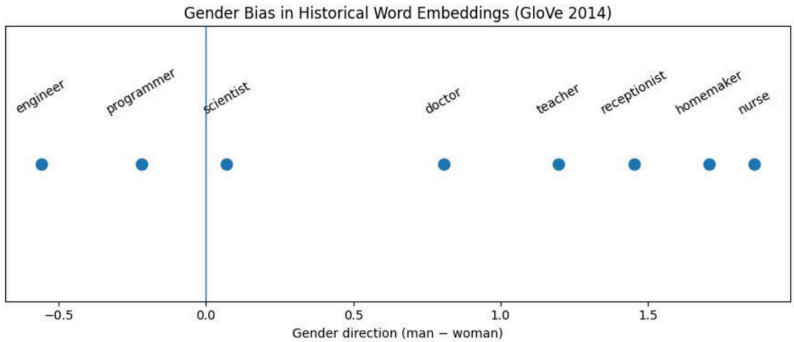
A summary of its training text includes the text's habits. We can find a "he–she" direction in our embedding (the differences between "he" and "she", "man" and "woman", "his" and "her", added up) and see where other words fall along it:



Shakespeare's "beauty", "fair" and "nurse" lean "she", his "soldier", "war" and "captain" lean "he". Nobody told the embedding anything about gender, it just noticed which words appear near "he" and which near "she" in plays written around 1600.

Bias in Modern Embeddings

Shakespeare's habits are four hundred years old. The same thing happens with embeddings trained on modern web text:



Pictured above is GloVe (2014), a widely used embedding trained on billions of words of the web, where "engineer" and "programmer" lean "man" and "homemaker" and "nurse" lean "woman". **A model built on these embeddings starts out with the same associations**, and carries them into everything it writes unless someone works to take them out.

Embedding a Prompt

Back to irishQwen. A language model doesn't look up one word at a time, it looks up a whole batch of prompts at once. A batch of B prompts of T tokens each is a tensor of ids of shape `(B, T)`, and the embedding layer turns it into a tensor of shape `(B, T, d)`, one row of d numbers for every token of every prompt:

```python
import torch
import torch.nn as nn
torch.manual_seed(0)
layer = nn.Embedding(512, 256)  # irishQwen, Shakespeare
x = torch.tensor([[446, 309, 44, 32, 275, 327, 294, 309],
                  [446, 327, 309, 44, 32, 275, 294, 309]])
out = layer(x)
print(x.shape, "->", out.shape)
print(out[0].sum(0)[:4])   # "To be, or not to be"
print(out[1].sum(0)[:4])   # "To not be, or to be"
```

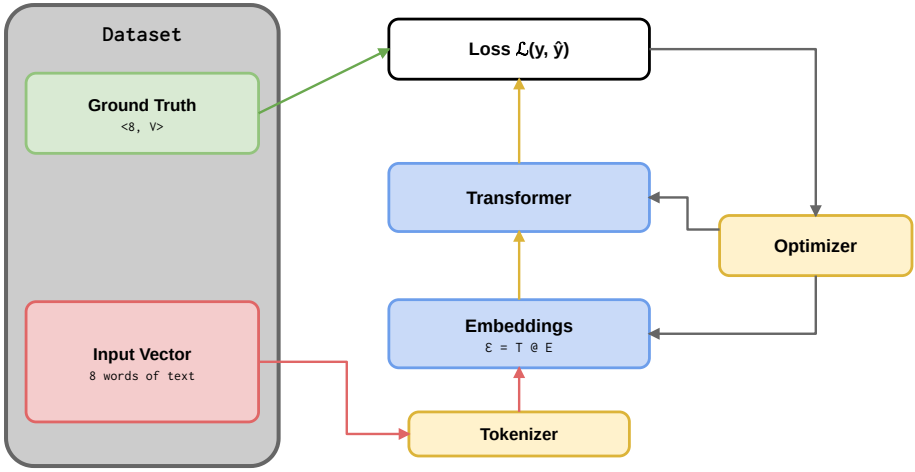
OUTPUT

```
torch.Size([2, 8]) -> torch.Size([2, 8, 256])
tensor([ 0.9530, -3.4446, -3.3353, -2.8090], grad_fn=<Backward>)
tensor([ 0.9530, -3.4446, -3.3353, -2.8090], grad_fn=<Backward>)
```

The first prompt is Hamlet's line in irishQwen's tokens, the second is the same eight tokens shuffled into "To not be, or to be". Each became **8** rows of **256** numbers. For the full Qwen3-0.6B a single prompt of T tokens becomes a tensor of shape `(1, T, 1024)`, and everything after the embedding layer works on that tensor.

Training a Text Prediction Model

irishQwen never runs word2vec. Its embedding layer is simply the first layer of the model, and its weights are trained the same way as everyone else's: the tokenizer turns text into ids, the embedding layer turns ids into vectors, the transformer turns vectors into a prediction of the next token, and the optimizer nudges every weight, embeddings included, to make the loss smaller.



Predicting the next token is a company-keeping task too (to predict a word you have to know what usually surrounds it), so **a trained language model ends up with embeddings much like word2vec's**, without ever being asked for them. Only the rows of tokens that appear in a batch get any gradient, which is why irishQwen's backward pass adds into just those columns of `W`.

Tied Embeddings

Qwen3-0.6B even uses its embedding table twice. Its output layer, which scores all **151,936** tokens as the possible next one, uses the same weights turned around, so a token's embedding is both how the model reads it and how the model says it. This is called **tied embeddings**.

At the end of the model each token has a vector **h** of **1,024** numbers, the model's summary of what should come next. With tied embeddings, the score for each possible next token is the dot product of **h** with that token's row of **E**:

one score per token in the vocabulary, <1, 151936>

$$\text{scores} = h @ E^{\mathsf{T}}$$

That is cosine similarity again, without the division by lengths. **The model predicts the token whose embedding points most nearly the way its summary points**, so everything in this chapter about directions applies to the model's output as much as to its input. Smaller Qwen3 models tie their embeddings to save weights (the table is a quarter of the model, as we saw), the bigger ones keep two separate tables.

Word Order

There is one more thing to notice about the output on page 27. The two prompts gave the same rows in a different order, and adding up the rows gave the same **256** numbers for both.

The embedding layer looks at every token on its own. " be" gets the same **256** numbers whether it comes second or eighth, and nothing in **ε** says which row came first. If a model just averaged the rows of a prompt it would be **unable to tell "to be or not to be" from "not to be or to be"**, and Hamlet would lose a good deal of his point.

Unfortunately order matters a lot for predicting the next token (the next word after "to be or not to" is "be", the next after "or not to be to" is anyone's guess). So the next step is a network that reads the rows of **ε** one at a time, in order, and keeps track of what it has read so far.